

FPGA ASSESSMENT 4

Anonymous public copy

Personal and institutional identifiers removed.

Contents

1 Problem Statement	3
1.1 Input/Output Specification	3
1.2 Mealy State Diagram	3
2 Verilog Code	3
3 Testbench	6
4 Simulation Output	7
5 FPGA Implementation Output	8
6 Conclusion	9

1 Problem Statement

The required design is a sequential circuit that receives a serial input X and produces two serial outputs $Y1$ and $Y0$. The circuit detects the number of 1's in the specified input sequences 10, 110, and 1110. For other longer sequences beginning with four or more consecutive 1's, such as 11110 and any other sequence of the form $111 \dots 110$, the output is 00. The design is to use a Mealy model and must include a reset input that returns the machine to the reset state.¹

1.1 Input/Output Specification

Input Sequence	Number of 1's ($Y1Y0$)
10	01
110	10
1110	11
11110 or any other sequence $111 \dots 110$	00

1.2 Mealy State Diagram

The minimum-state construction used by the supplied Verilog code has five states. $S0$ represents the reset/no-match condition, $S1$, $S2$, and $S3$ represent runs of one, two, and three consecutive 1's, respectively, and $S4$ represents four or more consecutive 1's. The outputs are associated with transitions, as required for a Mealy machine.

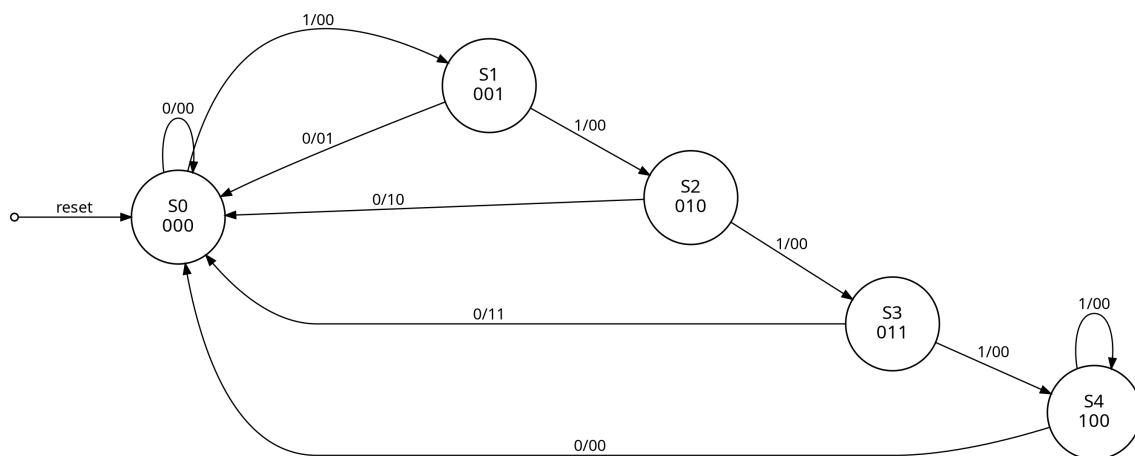


Figure 1: Mealy State Diagram

¹Question statement and required design constraints are reproduced from the supplied question PDF.

2 Verilog Code

```
1 module fsm(y0,y1,x,clk,rst);
2 output y0,y1;
3 input x,clk,rst;
4 reg y0,y1;
5
6 reg[2:0]cs,ns;
7
8 parameter s0=3'b000;
9 parameter s1=3'b001;
10 parameter s2=3'b010;
11 parameter s3=3'b011;
12 parameter s4=3'b100;
13
14 //cs logic
15
16 always@(posedge clk or posedge rst)
17
18 begin
19 if(rst)
20 cs<=s0;
21 else
22 cs<=ns;
23 end
24
25 //ns logic
26
27 always@(x or cs)
28 begin
29 case(cs)
30 s0:
31 begin
32 if(x)
33 ns=s1;
34 else
35 ns=s0;
36 end
37
38 s1:
39 begin
40 if(x)
41 ns=s2;
42 else
43 ns=s0;
44 end
45
46 s2:
47 begin
48 if(x)
49 ns=s3;
50 else
51 ns=s0;
52 end
53
54 s3:
55 begin
56 if(x)
57 ns=s4;
58 else
59 ns=s0;
60 end
61
62 s4:
63 begin
64 if(x)
65 ns=s4;
66 else
67 ns=s0;
68 end
69 endcase
70 end
71
72 //output logic
73
```

```

74  always@(x or cs)
75
76  begin
77
78      case(cs)
79
80          s0:
81              begin
82                  if(x)
83                      begin
84                          y1 = 1'b0;
85                          y0 = 1'b0;
86                      end
87                  else
88                      begin
89                          y1 = 1'b0;
90                          y0 = 1'b0;
91                      end
92                  end
93
94          s1:
95              begin
96                  if(x)
97                      begin
98                          y1 = 1'b0;
99                          y0 = 1'b0;
100                     end
101                 else
102                     begin
103                         y1 = 1'b0;
104                         y0 = 1'b1;
105                     end
106                 end
107
108          s2:
109              begin
110                  if(x)
111                      begin
112                          y1 = 1'b0;
113                          y0 = 1'b0;
114                      end
115                  else
116                      begin
117                          y1 = 1'b1;
118                          y0 = 1'b0;
119                      end
120                  end
121
122          s3:
123              begin
124                  if(x)
125                      begin
126                          y1 = 1'b0;
127                          y0 = 1'b0;
128                      end
129                  else
130                      begin
131                          y1 = 1'b1;
132                          y0 = 1'b1;
133                      end
134                  end
135
136          s4:
137              begin
138                  if(x)
139                      begin
140                          y1 = 1'b0;
141                          y0 = 1'b0;
142                      end
143                  else
144                      begin
145                          y1 = 1'b0;
146                          y0 = 1'b0;
147                      end
148                  end
149              endcase
150          end
151      endmodule

```

3 Testbench

```
1 module fsm_tb();
2
3 reg x,clk,rst;
4
5 wire y0,y1;
6
7 fsm fs1(y0,y1,x,clk,rst);
8
9
10 always #5 clk = ~clk;
11
12 initial
13 begin
14   clk =0;
15   rst =1;
16   x=0;
17
18   #10 rst =0;
19   #10 x =1;
20   #10 x =0;
21   #10 x =0;
22   #10 x =1;
23   #10 x =1;
24   #10 x =1;
25   #10 x =0;
26   #10 x =1;
27   #10 x =1;
28   #10 x =0;
29   #10 x =1;
30   #10 x =1;
31   #10 x =1;
32   #10 x =1;
33   #10 x =0;
34   #10 $stop;
35
36 end
37
38 endmodule
```

Test Sequence

Clock Cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Input X	1	0	0	1	1	1	0	1	1	0	1	1	1	1	0	-
Output Y1	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	-
Output Y0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	-

4 Simulation Output

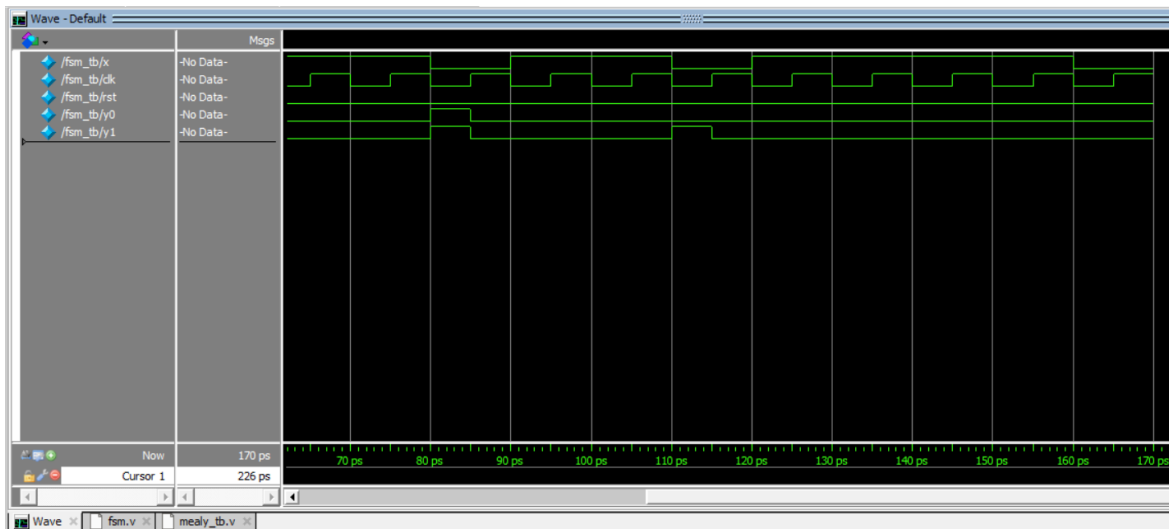


Figure 2: Mealy State Diagram

5 FPGA Implementation Output

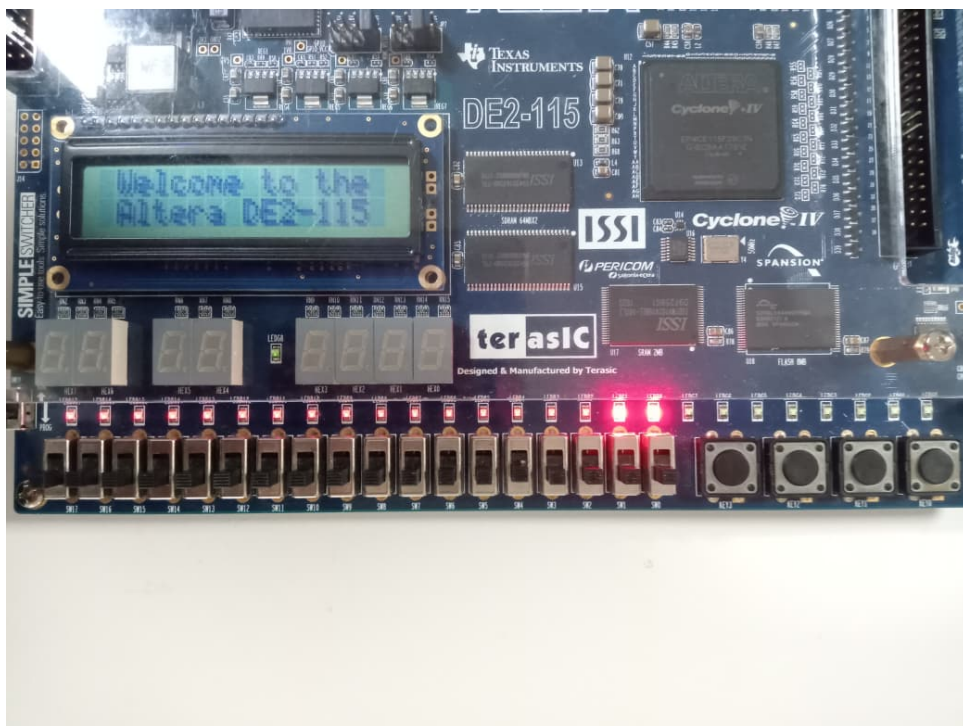


Figure 3: Output at 6th clock

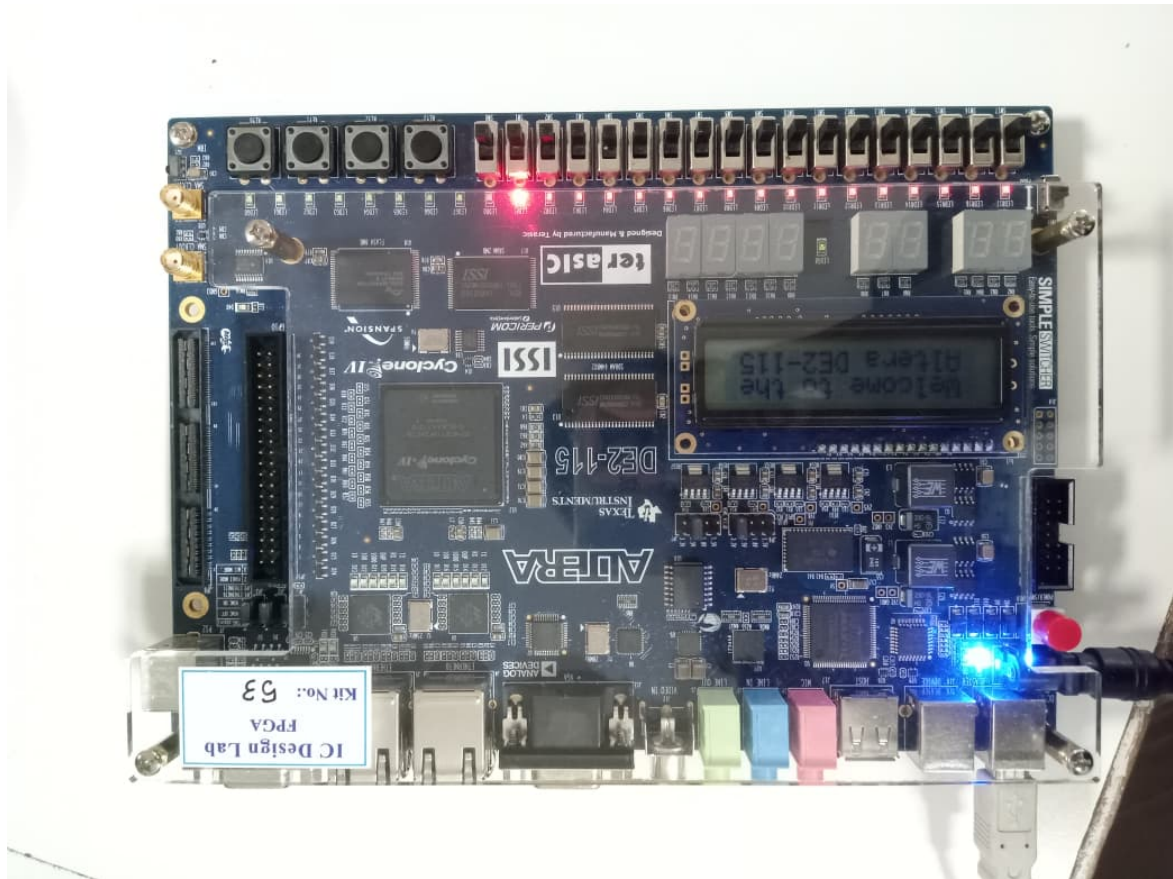


Figure 4: Output at 7th clock

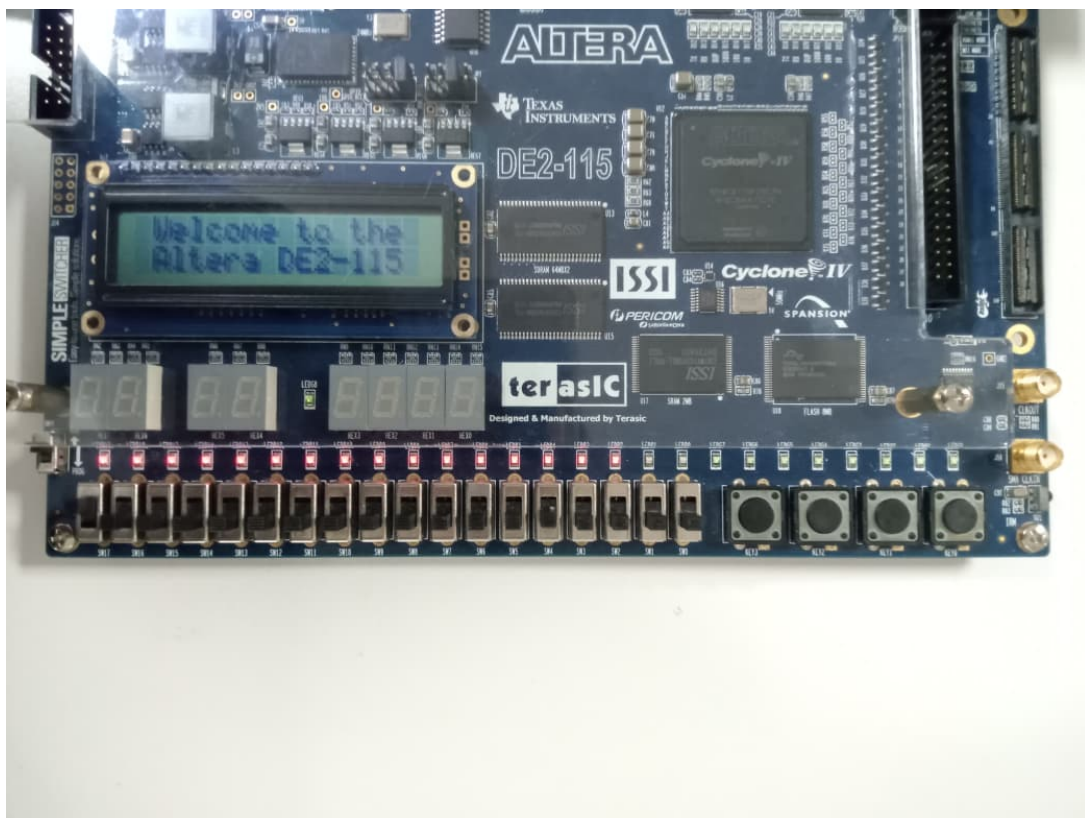


Figure 5: Reset state

FPGA Pin Assignment

```
set_location_assignment PIN_G19 -to y0
```

```
set_location_assignment PIN_F19 -to y1
```

```
set_location_assignment PIN_AB28 -to x
```

```
set_location_assignment PIN_Y23 -to rst
```

```
set_location_assignment PIN_M23 -to clk
```

6 Conclusion

A five-state Mealy finite state machine was used to detect the required serial patterns. The design separates present-state storage, next-state logic, and output logic, and the reset input returns the machine to *S0*. The supplied testbench exercises the input stream specified for the assessment and provides a reference for simulation verification. The FPGA section is intentionally formatted with dedicated photo spaces for the final hardware evidence.